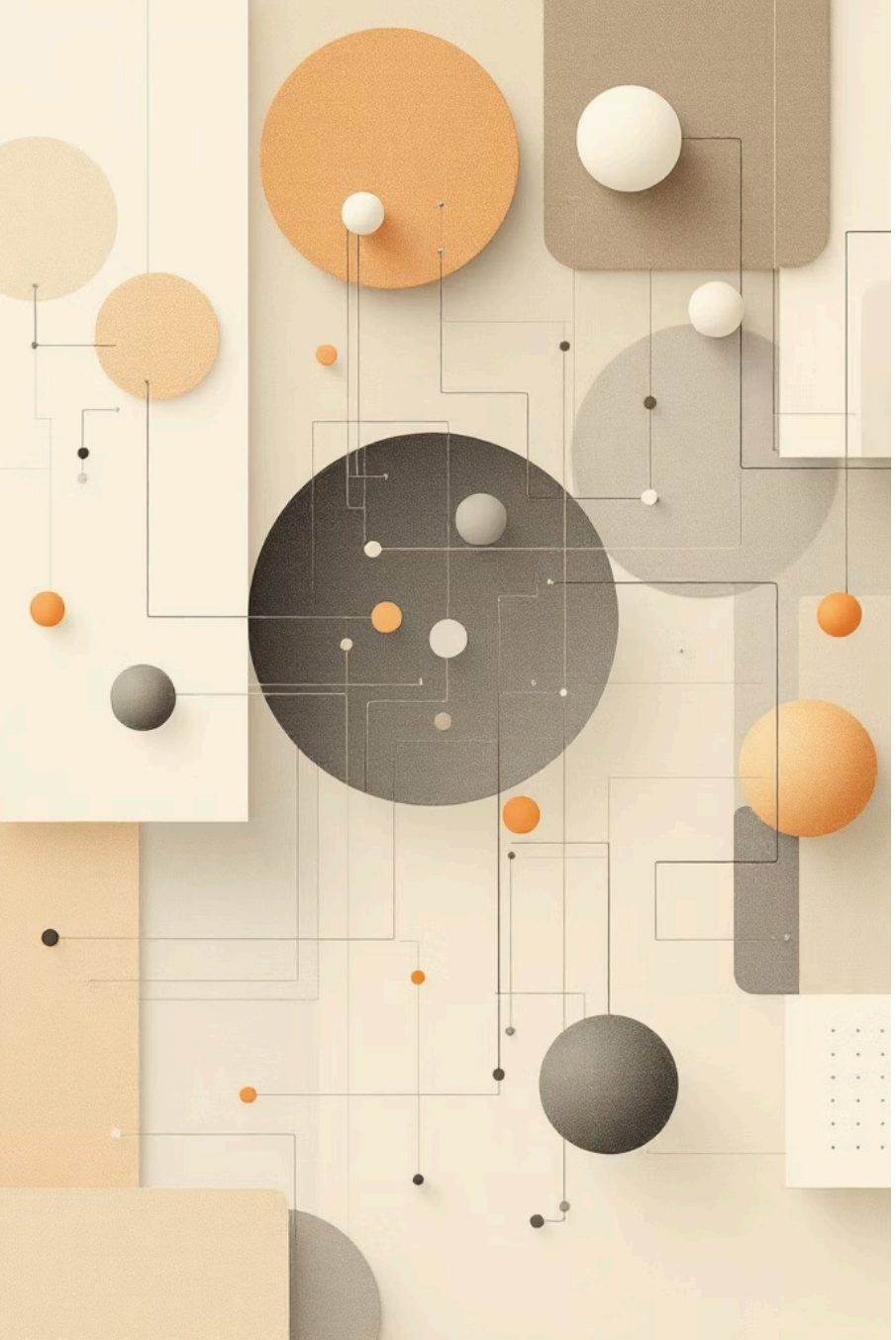


Assignment Problem Using Brute Force Method

Algorithm Design & Analysis — A foundational look at how exhaustive search solves optimisation problems

- **Avyakta Bhat (1RN24CY006)**
- **Prashant Gourav (1RN24CY034)**
- **Mohan Murari Sharma (1RN24CY026)**

What Is the Assignment Problem?

The Core Challenge

Assign **n agents** to **n tasks** such that the total cost is minimised — and each agent handles exactly one task.

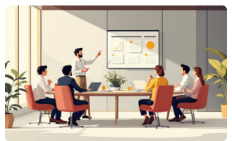
Why It Matters

A fundamental problem in **operations research** and computer science, appearing wherever limited resources must be matched efficiently.

The Constraint

Every agent is assigned to **exactly one task**, and every task is covered — a strict one-to-one mapping.

Real-Life Applications



Job Assignment

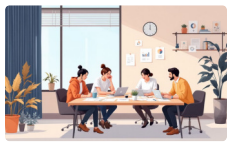
Matching employees to roles in companies based on skill fit and cost.

Task Scheduling

Operating systems allocating processes to CPU cores efficiently.

Machine-Task Allocation

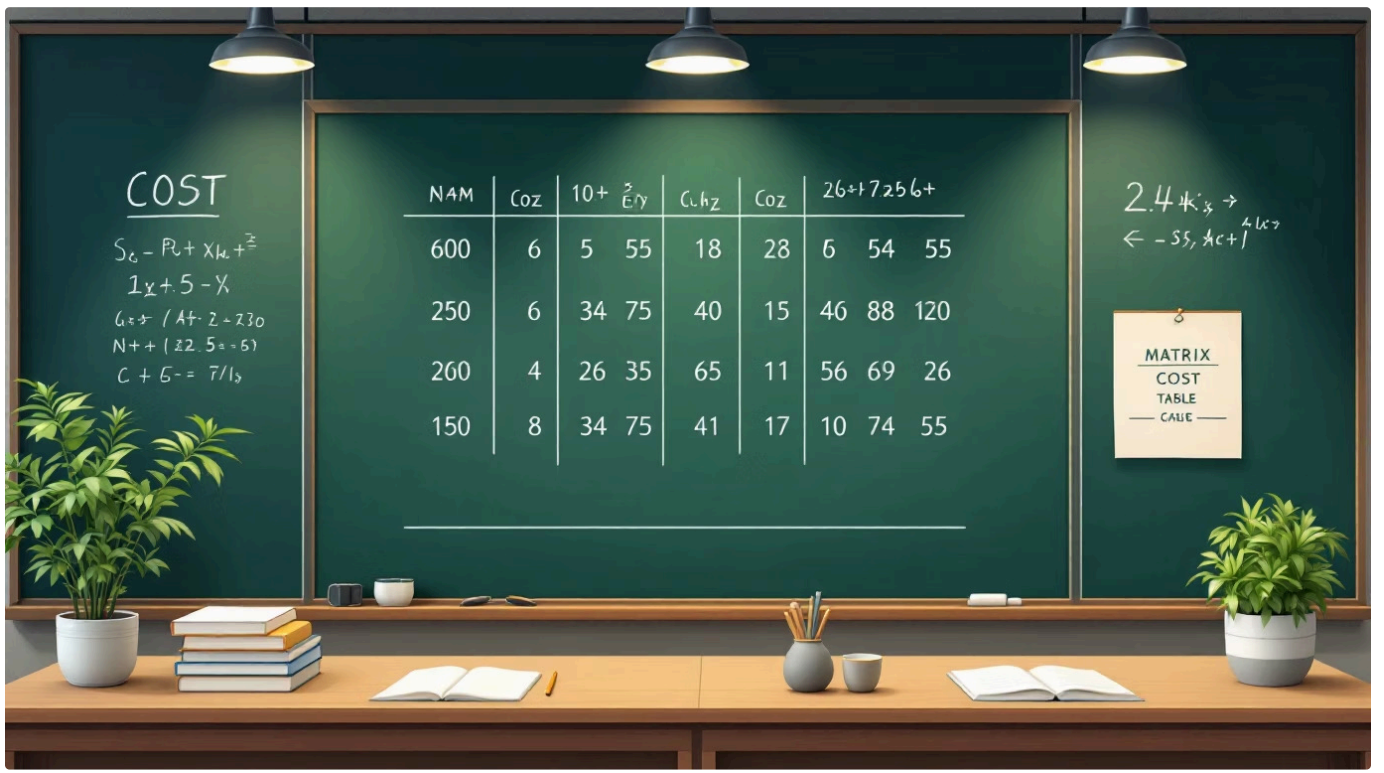
Assigning jobs to machines in manufacturing to minimise downtime.



Team Allocation

Assigning people to projects based on availability and expertise.

Problem Definition



Inputs

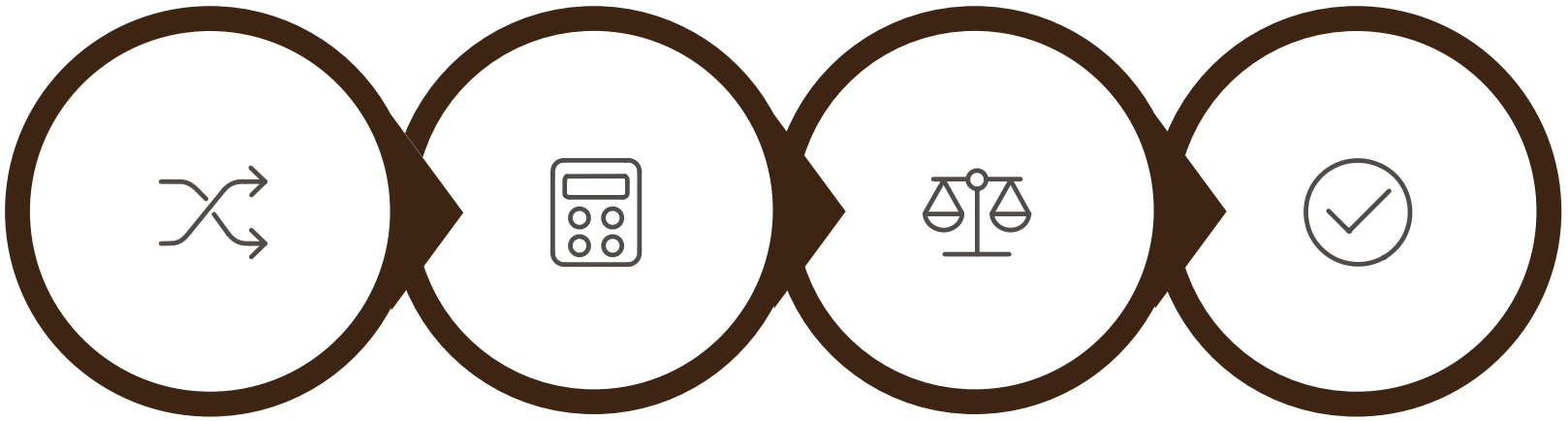
- **n agents** and **n tasks**
- A cost matrix of size **n × n**
- Each element **C[i][j]** = cost of assigning task *j* to agent *i*

Goal

Find the one-to-one assignment that produces the **minimum total cost** across all agent-task pairs.

Constraint: No two agents share a task, and no task is left unassigned.

The Brute Force Workflow



Brute force leaves nothing to chance — it exhaustively evaluates every possible assignment before declaring the optimal one. Simple in concept, but demanding in execution.

01

Generate Permutations

Create every possible one-to-one assignment of tasks to agents.

02

Compute Total Cost

Calculate the full assignment cost for each permutation.

03

Compare Costs

Review all computed totals and identify the smallest values.

04

Select Minimum

Choose the assignment with the lowest overall cost.

Numerical Example: Cost Matrix

This cost matrix shows the assignment cost for each agent-task pair. Next, we'll evaluate all possible assignments to find the minimum total cost.

	Task 1	Task 2	Task 3
Agent 1	10	12	8
Agent 2	7	9	11
Agent 3	15	6	13

Evaluating All Permutations

Brute force evaluates all 6 permutations, and the table below shows how the task assignments change for each agent across each permutation. **Permutation 5 has the minimum cost of 21.**

Permutation	Agent 1	Agent 2	Agent 3	Total Cost
Permutation 1	Task 1 (10)	Task 2 (9)	Task 3 (13)	10 + 9 + 13 = 32
Permutation 2	Task 1 (10)	Task 3 (11)	Task 2 (6)	10 + 11 + 6 = 27
Permutation 3	Task 2 (12)	Task 1 (7)	Task 3 (13)	12 + 7 + 13 = 32
Permutation 4	Task 2 (12)	Task 3 (11)	Task 1 (15)	12 + 11 + 15 = 38
Permutation 5	**Task 3 (8)**	**Task 1 (7)**	**Task 2 (6)**	**8 + 7 + 6 = 21**
Permutation 6	Task 3 (8)	Task 2 (9)	Task 1 (15)	8 + 9 + 15 = 32

Time Complexity: O(n!)

6

Permutations at n=3

Trivially fast — manageable by hand.

120

Permutations at n=5

Still computable, but growing quickly.

3.6M

Permutations at n=10

Over 3.6 million evaluations required.

The number of permutations equals **n!**, meaning complexity grows **factorially**. Even modest increases in *n* make brute force computationally infeasible.

Advantages & Limitations

Advantages

- Simple and intuitive — easy to understand and implement
- Guarantees the **globally optimal** solution
- Requires no complex data structures or logic

Limitations

- Grows at **O(n!)** — impractical beyond small inputs
- Not scalable for real-world problem sizes
- High memory and CPU overhead for large *n*

Conclusion

→ Brute Force Works — But Has a Cost

It checks every permutation, guaranteeing the best answer for small datasets.

→ Factorial Complexity Is the Bottleneck

O(n!) makes it impractical for *n* beyond ~10 in most real applications.

→ Better Alternatives Exist

The **Hungarian Algorithm** solves the same problem in $\mathbf{O(n^3)}$ — a dramatic improvement for large inputs.